

Тема 4.3 Сокеты. Клиентские сокеты. Серверные сокеты

Сокеты. Создание клиента

Сокет — это программный интерфейс для обеспечения информационного обмена между процессами.

Виды сокетов:

- серверный - сокет, который принимает сообщения;
- клиентский - сокет, который отправляет сообщения.

Сокеты работают на транспортном уровне протокола и соответственно бывают 2 типов:

- потоковые (на основе TCP, в коде обозначаются SOCK_STREAM) - сокеты с установленным соединением на основе протокола TCP, передают поток байтов, который может быть двунаправленным - т.е. приложение может и получать и отправлять данные;
- дейтаграммные (на основе UDP, в коде обозначаются SOCK_DGRAM) - сокеты, не требующие установления явного соединения между ними. Сообщение отправляется указанному сокету и, соответственно, может получаться от указанного сокета.

Сокет состоит из IP-адреса и порта.

IP-адрес - уникальный сетевой адрес узла в компьютерной сети, построенной по протоколу IP. В версии протокола IPv4 IP-адрес имеет длину 4 байта (например, 192.168.0.3), а в версии протокола IPv6 IP-адрес имеет длину 16 байт (например, 2001:0db8:85a3:0000:0000:8a2e:0370:7334). IP-адрес должен быть уникален.

Порт - натуральное число, записываемое в заголовках протоколов транспортного уровня (TCP, UDP и др.). Порт используется для определения процесса-получателя пакета в пределах одного хоста.

Сокет представляет собой программный интерфейс, который обеспечивает двустороннюю связь между узлами в компьютерной сети. Это виртуальный разъём, через который приложения могут отправлять и получать данные по сети.

Современные сетевые приложения активно используют сокеты Python для создания различных систем. От мессенджеров до онлайн-игр и сервисов обмена данными — везде применяется сетевое взаимодействие через сокеты.

Для работы с сокетами в Python необходимо понимать ключевые концепции:

- IP-адрес — уникальный адрес устройства в сети;
- Порт — числовой идентификатор сервиса на устройстве;
- TCP (Transmission Control Protocol) — надёжный протокол передачи данных;
- UDP (User Datagram Protocol) — быстрый протокол без подтверждения доставки;
- TCP обеспечивает гарантированную доставку данных с контролем ошибок. UDP работает быстрее, но не гарантирует доставку сообщений.

Функции объекта socket:

- `socket.bind(address)` - Привязывает сокет к адресу `address` (инициализирует IP-адрес и порт). Сокет не должен быть привязан до этого;
- `socket.listen([backlog])` - Переводит сервер в режим приема соединений. Параметр `backlog (int)` – количество соединений, которые будет принимать сервер;
- `socket.accept()` - Принимает соединение и блокирует приложение в ожидании сообщения от клиента. В результате возвращает кортеж:
 - `conn`: объект соединения (сокет), который можно использовать для отправки/получения данных;
 - `address`: адрес клиента.
- `socket.recv(bufsize[, flags])` - Читает и возвращает данные в двоичном формате (набор байтов) из сокета. Параметр `bufsize (int)` – максимальное количество байтов в одном сообщении;
- `socket.send(bytes[, flags])` - Отправляет данные клиенту и возвращает количество отправленных байт. Параметр `bytes (bytes)` – двоичные данные;
- `socket.close()` - Закрывает сокет.

Работа с сокетом во многом схожа с работой с файловым объектом. Принцип - открыли соединение - считали данные - закрыли соединение.

Модуль `socket` входит в стандартную библиотеку Python. Для начала работы с сокетами достаточно выполнить простой импорт:

```
import socket
```

Вне зависимости, что мы создаем - программу сервера или клиента, для отправки запросов нам надо создать объект сокета:

```
import socket
sock = socket.socket()
```

После окончания работы с сокетом его следует закрыть с помощью метода `close()`.

```
import socket
client = socket.socket()
client.close()
```

Дальнейшие действия с сокетом зависят от того, какой именно сокет мы создаем - для сервера или для клиента. В данном случае мы рассмотрим создание простейшего клиента на сокетах Python.

Для подключения к серверу применяется метод `connect()`.

```
socket.connect(address)
```

В качестве параметра он получает адрес сервера. Адрес обычно представляется в виде кортежа

```
(host, port)
```

Первый элемент - хост в виде строки. Это может быть, например, IP-адрес в виде "127.0.0.1" или название локального хоста. Второй параметр - числовой номер

```
import socket
client = socket.socket()
# подключаемся по адресу www.python.org и порту 80
```

```
client.connect(("www.python.org", 80))
print("Connected...")
client.close() # закрываем подключение
```

Здесь пытаемся подключиться по адресу "www.python.org" и порту 80. То есть фактически мы пытаемся подключиться к обычному веб-сайту www.python.org, который работает на порту 80 (обычно веб-сервер запускается на порту 80). После подключения просто выводим строку на консоль и закрываем сокет.

После установки соединения мы можем отправлять и получать данные от сервера. Для отправки данных применяется метод `socket.send()`, который в качестве параметра получает набор отправляемых данных.

```
socket.send(bytes)
```

Для получения данных у сокета применяется метод `socket.recv()`

```
bytes = socket.recv(bufsize[, flags])
```

В качестве обязательного параметра он принимает максимальный размер буфера в байтах, которые могут быть получены за раз от другого сокета. Размер буфера лучше делать кратным 2, например, 512, 1024 и т.д. Возвращаемое значение - набор байтов, полученных от другого сокета.

Например, отправим данные на сервер "www.python.org" и получим от него ответ:

```
import socket
client = socket.socket()
# подключаемся по адресу www.python.org и порту 80
client.connect(("www.python.org", 80))
# данные для отправки - стандартные заголовки протокола http
message = "GET / HTTP/1.1\r\nHost: www.python.org\r\nConnection:
close\r\n\r\n"
print("Connecting...")
client.send(message.encode()) # отправляем данные серверу
data = client.recv(1024) # получаем данные с сервера
print("Server sent: ", data.decode()) # выводим данные на консоль
client.close() # закрываем подключение
```

В данном случае отправляется строка со стандартными заголовками протокола HTTP, которые понимает веб-сайт. Формат запроса HTTP включает прежде всего линию запроса ("GET / HTTP/1.1\r\nHost: www.python.org"), которая состоит из типа запроса, пути к запрошенному ресурсу и специфической версии протокола. То есть здесь в сообщении мы указываем, что отправляется запрос типа GET по пути "/" (то есть к корню сайта www.python.org). При этом применяется протокол HTTP/1.1. Линия запроса должна завершаться двойным набором символов каретки и перевода строки `\r\n`.

Кроме того, запрос HTTP может содержать заголовки. Так, в данном случае отправляем заголовок "Host", который указывает на адрес хоста. В данном случае это "www.python.org:80". И также в данном случае отправляем заголовок "Connection", который имеет значение "close" — это значение предписывает серверу закрыть подключение.

Поскольку мы можем послать только байты, а не строки, то при отправке переводим строку в набор байтов. Для этого применяется метод `encode()`, который возвращает объект класса `bytes`:

```
client.send(message.encode())
```

После отправки серверу сообщения пытаемся получить от него данные с помощью метода `recv()`. При этом буфер для получаемых данных будет иметь размер в 1024 байта. Результат метода - полученные данные. Однако сокет получает данные в виде набора байт (точнее объекта `bytes`). Чтобы их преобразовать в строку, применяется метод `decode()`:

```
data = client.recv(1024) # получаем данные с сервера
print("Server sent: ", data.decode()) # выводим данные на консоль
```

И если мы запустим данную программу на Python, то получим ответ наподобие следующего:

```
Connected...
Server sent: HTTP/1.1 301 Moved Permanently
Connection: close
Content-Length: 0
Server: Varnish
Retry-After: 0
Location: https://www.python.org/
Accept-Ranges: bytes
Date: Tue, 02 May 2023 17:52:32 GMT
Via: 1.1 varnish
X-Served-By: cache-bma1653-BMA
X-Cache: HIT
X-Cache-Hits: 0
X-Timer: S1683049953.637569,VS0,VE0
Strict-Transport-Security: max-age=63072000; includeSubDomains; preload
Сервер также нам присылает строку в http-заголовках, которые
указывают, что веб-сайт переехал на адрес https://www.python.org/.
```

Пример реализации клиента:

```
import socket
client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM) #
Создание клиентского сокета
client_socket.connect(('localhost', 12345)) # Подключение к серверу
client_socket.send("Привет, сервер!".encode('utf-8')) # Отправка сообщения
response = client_socket.recv(1024).decode('utf-8') # Получение ответа
print(f"Ответ от сервера: {response}")
client_socket.close() # Закрытие соединения
```

Сокеты. Создание сервера

Для создания сервера на языке Python, как и для клиента, также используется класс `socket`, однако общая работа будет несколько отличаться от

работы с сокетом клиента. Рассмотрим определение и работу с сокетом сервера поэтапно.

Сервер прослушивает входящие подключения, некоторым образом обрабатывает их и отправляет ответ. Чтобы сервер начал свою работу, вначале нам надо определить для него адрес, по которому он будет прослушивать подключения. Для этого применяется метод `bind()`

```
socket.bind(address)
```

Данный метод принимает адрес, по которому будет запущен сервер. По умолчанию адрес представляет кортеж из двух элементов:

```
(host, port)
```

Первый элемент - хост в виде строки. Это может быть, например, IP-адрес в виде "127.0.0.1" или название локального хоста. Второй параметр - числовой номер порта. Порт представляет 2-х байтное значение от 0 до 65535. Поскольку по одному и то же адресу (на одной и той же машине) может быть запущено несколько различных сетевых приложений, то порт позволяет разграничить эти приложения. Например:

```
import socket
server = socket.socket() # создаем объект сокета сервера
hostname = socket.gethostname() # получаем имя хоста локальной машины
port = 12345 # устанавливаем порт сервера
server.bind((hostname, port)) # привязываем сокет сервера к хосту и порту
```

Здесь сервер будет запускаться на порту 12345. Следует учитывать, что не все порты могут быть свободны. Но, как правило, занятых портов не так много.

В качестве адреса используем имя текущего хоста. Для его получения применяется функция `socket.gethostname()` (обычно это имя текущего компьютера).

После привязки сервера его надо запустить на прослушивание подключений. Для этого применяется метод `listen()`

```
socket.listen([backlog])
```

Он принимает параметр `backlog` - максимальное количество входящих подключений в очереди, разрешенное для сокета. То есть, когда будут подключаться клиенты, они будут попадать в очередь и ждать, пока сервер не обработает текущего клиента. Если в очереди уже есть указанное количество клиентов, ожидающих обработки сервером, то все новые клиенты отклоняются. Например:

```
import socket
server = socket.socket() # создаем объект сокета сервера
hostname = socket.gethostname() # получаем имя хоста локальной машины
port = 12345 # устанавливаем порт сервера
server.bind((hostname, port)) # привязываем сокет сервера к хосту и порту
server.listen(5) # начинаем прослушивание входящих подключений
```

Для получения входящих подключений применяется метод `accept()`. Этот метод возвращает кортеж из двух элементов

```
(conn, address)
```

Первый элемент - `conn` представляет еще один объект `socket`, через который сервер взаимодействует с клиентом. Второй элемент - `address` - адрес подключившегося клиента. Стоит отметить, что после завершения взаимодействия с клиентом сокет `conn` надо закрыть методом `close()`

Используя первый элемент кортежа - `conn` можно отправлять клиенту сообщения или наоборот получать данные. Для получения данных у сокета применяется метод `socket.recv()`

```
bytes = socket.recv(bufsize)
```

В качестве обязательного параметра он принимает максимальный размер буфера в байтах, которые могут быть получены за раз от другого сокета. Возвращаемое значение - набор байтов, полученных от другого сокета.

Для отправки данных применяется метод `socket.send()`, который в качестве параметра получает набор отправляемых данных.

```
socket.send(bytes)
```

Теперь посмотрим все на примере. Пусть в файле `server.py` будет определен сервер со следующим кодом:

```
import socket
server = socket.socket() # создаем объект сокета сервера
hostname = socket.gethostname() # получаем имя хоста локальной машины
port = 12345 # устанавливаем порт сервера
server.bind((hostname, port)) # привязываем сокет сервера к хосту и порту
server.listen(5) # начинаем прослушивание входящих подключений
print("Server starts")
con, addr = server.accept() # принимаем клиента
print("connection: ", con)
print("client address: ", addr)
message = "Hello Client!" # сообщение для отправки клиенту
con.send(message.encode()) # отправляем сообщение клиенту
con.close() # закрываем подключение
print("Server ends")
server.close()
```

Здесь сервер принимает клиента, выводит информацию о его подключении и отправляет клиенту в ответ строку "Hello Client!".

А в файле `client.py` определим следующий код клиента:

```
import socket
client = socket.socket() # создаем сокет клиента
hostname = socket.gethostname() # получаем хост сервера
port = 12345 # устанавливаем порт сервера
client.connect((hostname, port)) # подключаемся к серверу
data = client.recv(1024) # получаем данные с сервера
print("Server sent: ", data.decode()) # выводим данные на консоль
client.close() # закрываем подключение
```

Поскольку у нас сервер и клиент будут запускаться на одном и том же компьютере, то для определения адреса сервера для подключения применяется

функция `socket.gethostname()` и порт 12345 - так же как и в коде сервера. После соединения с сервером получаем от него данные и выводим на консоль.

Сначала запустим код сервера. А затем запустим код клиента. В результате сервер примет подключение, выведет информацию о нем на консоль и отправит клиенту сообщение:

```
c:\python>python server.py
Server starts
connection:      <socket.socket fd=432, family=2, type=1, proto=0,
laddr=('192.168.0.102', 12345), raddr=('192.168.0.102', 61824)>
client address: ('192.168.0.102', 61824)
Server ends
```

В частности, здесь видим, что сервер и клиент запущены по адресу 192.168.0.102, причем клиент использует порт 61824.

А клиент получит от сервера сообщение и выведет его на консоль:

```
c:\python>python client.py
Server sent: Hello Client!
```

В данном случае сервер обслуживает одного клиента и прекращает работу. Если мы хотим, чтобы сервер обрабатывал множество клиентов, то мы можем использовать бесконечный цикл:

```
import socket
from datetime import datetime
server = socket.socket() # создаем объект сокета сервера
hostname = socket.gethostname() # получаем имя хоста локальной машины
port = 12345 # устанавливаем порт сервера
server.bind((hostname, port)) # привязываем сокет сервера к хосту и порту
server.listen(5) # начинаем прослушивание входящих подключений
print("Server running")
while True:
    con, addr = server.accept() # принимаем клиента
    print("client address: ", addr)
    message = datetime.now().strftime("%H:%M:%S") # отправляем текущее
    время
    con.send(message.encode()) # отправляем сообщение клиенту
    con.close() # закрываем подключение
```

В данном случае для примера с помощью встроенного модуля `datetime` и функции `datetime.now().strftime()` получаем текущее время в виде строки, которая затем отправляется клиенту. В итоге при запросе клиент будет получать текущее время.

В примерах выше связь была однонаправленная - сервер отправлял данные, а клиент получал их. Рассмотрим простейшую двунаправленную связь, когда и клиент, и сервер и отправляют, и получают данные. Пусть сервер получает от клиента некоторую строку, инвертирует ее и отправляет обратно клиенту:

```
import socket
server = socket.socket() # создаем объект сокета сервера
hostname = socket.gethostname() # получаем имя хоста локальной машины
```

```

port = 12345                # устанавливаем порт сервера
server.bind((hostname, port)) # привязываем сокет сервера к хосту и порту
server.listen(5)            # начинаем прослушивание входящих подключений
print("Server running")
while True:
    con, _ = server.accept() # принимаем клиента
    data = con.recv(1024)    # получаем данные от клиента
    message = data.decode()  # преобразуем байты в строку
    print(f"Client sent: {message}")
    message = message[::-1]  # инвертируем строку
    con.send(message.encode()) # отправляем сообщение клиенту
    con.close()              # закрываем подключение

```

А клиент пусть определяет код для ввода строки с консоли и ее отправки на сервер:

```

import socket
client = socket.socket() # создаем сокет клиента
hostname = socket.gethostname() # получаем хост локальной машины
port = 12345 # устанавливаем порт сервера
client.connect((hostname, port)) # подключаемся к серверу
message = input("Input a text: ") # вводим сообщение
client.send(message.encode()) # отправляем сообщение серверу
data = client.recv(1024) # получаем данные с сервера
print("Server sent: ", data.decode())
client.close() # закрываем подключение

```

Результат работы. Клиент:

```
c:\python>python client.py
```

```
Input a text: hello
```

```
Server sent: olleh
```

```
c:\python>
```

Сервер:

```
c:\python>python server.py
```

```
Server running
```

```
Client sent: hello
```

TCP-сервер принимает соединения от клиентов и обрабатывает их запросы.

Рассмотрим простую реализацию:

```

import socket
# Создание сокета TCP
server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
# Привязка к адресу и порту
server_socket.bind(('localhost', 12345))
# Начало прослушивания (максимум 5 подключений в очереди)
server_socket.listen(5)
print("Сервер запущен. Ожидание подключений...")
while True:
    # Принятие подключения

```



```

client_socket, address = server_socket.accept()
print(f"Подключение от {address}")
# Получение сообщения
message = client_socket.recv(1024).decode('utf-8')
print(f"Получено сообщение: {message}")
# Отправка ответа
client_socket.send("Привет от сервера!".encode('utf-8'))
# Закрытие соединения с клиентом
client_socket.close()

```

В приведенном коде происходят следующие операции:

- создается сокет с использованием протокола TCP;
- сервер привязывается к адресу localhost и порту 12345;
- запускается ожидание подключений клиентов;
- при подключении клиента принимается его сообщение;
- сервер отправляет ответ и закрывает соединение.

Метод `socket.AF_INET` указывает на использование IPv4, а `socket.SOCK_STREAM` — на протокол TCP.

Многопоточное клиент-серверное приложение

Многопоточность позволяет одновременно выполнять несколько различных действий в различных потоках. Применение многопоточности в серверном приложении позволяет обрабатывать одновременно несколько клиентов. Рассмотрим, как создать многопоточное клиент-серверное приложение.

В Python многопоточность обеспечивается функциональностью модулей `_thread` и `threading`. В частности, для запуска нового потока мы можем использовать функцию `start_new_thread()` из модуля `_thread`, которая имеет следующее определение:

```
_thread.start_new_thread(function, args[, kwargs])
```

Эта функция запускает поток, который выполняет функцию из первого параметра `function`, передавая ей в качестве аргументов кортеж `args`. Опционально можно передать словарь дополнительных параметров через третий параметр `kwargs`

Например, определим в файле `server.py` следующий код сервера

```

import socket
from _thread import *
# функция для обработки каждого клиента
def client_thread(con):
    data = con.recv(1024) # получаем данные от клиента
    message = data.decode() # преобразуем байты в строку
    print(f"Client sent: {message}")
    message = message[::-1] # инвертируем строку
    con.send(message.encode()) # отправляем сообщение клиенту
    con.close() # закрываем подключение
server = socket.socket() # создаем объект сокета сервера

```

```

hostname = socket.gethostname() # получаем имя хоста локальной машины
port = 12345 # устанавливаем порт сервера
server.bind((hostname, port)) # привязываем сокет сервера к хосту и порту
server.listen(5) # начинаем прослушивание входящих подключений
print("Server running")
while True:

```

```

    client, _ = server.accept() # принимаем клиента
    start_new_thread(client_thread, (client,)) # запускаем поток клиента

```

Здесь при подключении каждого нового подключения функция `start_new_thread()` запускает для его обработки функцию `client_thread()` и передает ей текущего клиента, который хранится в переменной `client`. В функции `client_thread()` для примера получаем от клиента строку, инвертируем ее и отправляем обратно клиенту.

Для тестирования сервера определим следующий код клиента:

```

import socket
client = socket.socket() # создаем сокет клиента
hostname = socket.gethostname() # получаем хост локальной машины
port = 12345 # устанавливаем порт сервера
client.connect((hostname, port)) # подключаемся к серверу
message = input("Input a text: ") # вводим сообщение
client.send(message.encode()) # отправляем сообщение серверу
data = client.recv(1024) # получаем данные с сервера
print("Server sent: ", data.decode())
client.close() # закрываем подключение

```

Клиент ожидает ввод с консоли строки, которая отправляется серверу.

Ответ сервера выводится на консоль.

Запустим сервер, затем запустим клиент. Пример работы. Сервер:

```
c:\python>python server.py
```

```
Server running
```

```
Client sent: hello
```

Клиент:

```
c:\python>python client.py
```

```
Input a text: hello
```

```
Server sent: olleh
```

```
c:\python>
```

Отправка файлов

Довольно часто в клиент-серверных приложениях встречается задача отправки и получения файлов. Рассмотрим, как это сделать. Допустим, у нас есть файл `hello.txt` с простейшим содержимым:

```
hello world
```

```
hello test
```

Для отправки файла определим в файле `server.py` следующий код сервера:

```

import socket
server = socket.socket()      # создаем объект сокета сервера
hostname = socket.gethostname() # получаем имя хоста локальной
машины
port = 12345                  # устанавливаем порт сервера
server.bind((hostname, port)) # привязываем сокет сервера к хосту и
порту
server.listen(5)              # начинаем прослушивание входящих
подключений
print("Server running")
con, _ = server.accept()      # принимаем клиента
filename="hello.txt"         # имя файла для отправки
file = open(filename, "rb")   # открываем файл для отправки
print("sending data to client")
# считываем данные из файла блоками по 1024 байт и отправляем клиенту
line = file.read(1024)
while(line):
    con.send(line)            # отправляем строку клиенту
    line = file.read(1024)
file.close()                  # закрываем файл
con.close()                   # закрываем клиента
server.close()

```

Здесь открываем файл в бинарном режиме, считываем его блоками по 1024 байта и в цикле отправляем клиенту, пока не считаем и не отправим все данные из файла.

Для получения файла в файле client.py определим следующий код клиента:

```

import socket
client = socket.socket()      # создаем сокет клиента
hostname = socket.gethostname() # получаем хост локальной машины
port = 12345                  # устанавливаем порт сервера
client.connect((hostname, port)) # подключаемся к серверу
print("receiving data from server")
while True:
    data = client.recv(1024)   # получаем данные от сервера
    print(bytes.decode(data))
    if not data: break
client.close()

```

Здесь в бесконечном цикле получаем от сервера данные блоками по 1024 байта и выводим их на консоль. Когда данных больше не будет, выходим из цикла.

Запустим сервер и затем клиент. Консоль сервера отобразит следующее:

```
c:\python>python server.py
```

```
Server running
```

```
sending data to client
```

А консоль клиента получит данные файла и выведет их на консоль:

```
c:\python>python client.py
```

```
receiving data from server
hello world
hello test
```

Мы можем пойти дальше и вместо вывода полученного содержимого файла на консоль сохранять файл на клиенте:

```
import socket
client = socket.socket()          # создаем сокет клиента
hostname = socket.gethostname()   # получаем хост локальной машины
port = 12345                      # устанавливаем порт сервера
client.connect((hostname, port))  # подключаемся к серверу
file = open("test.txt", "wb")
print("receiving data from server")
while True:
    data = client.recv(1024)      # получаем данные от сервера
    file.write(data)              # записываем данные в файл
    if not data: break
file.close()                     # закрываем файл
print("data saved")
client.close()
```

В данном случае полученные данные сохраняются в файл "test.txt", который будет создаваться в одной папке со скриптом клиента.

Пример решения задачи на языке программирования Python

Задача. Напишите клиент-серверное приложение, в котором клиент отправляет серверу строку текста. Сервер принимает сообщение, преобразует его в верхний регистр и отправляет результат обратно клиенту. Клиент должен вывести полученный ответ на экран.

Решение задачи:

Сервер (server.py)

```
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
server.bind(('localhost', 5000))
server.listen(1)
print("Сервер запущен. Ожидание подключения...")
conn, addr = server.accept()
print("Подключен клиент:", addr)
message = conn.recv(1024).decode()
print("Получено:", message)
response = message.upper()
conn.send(response.encode())
conn.close()
server.close()
```

Клиент (client.py)

```
import socket
client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
client.connect(('localhost', 5000))
message = input("Введите сообщение: ")
client.send(message.encode())
response = client.recv(1024).decode()
print("Ответ сервера:", response)
client.close()
```

Работа с UDP сокетами в Python

UDP (User Datagram Protocol) — транспортный протокол, обеспечивающий передачу данных без установления соединения.

В отличие от TCP, UDP является протоколом без установления соединения (connectionless). Это означает, что перед отправкой данных не требуется устанавливать специальное соединение между клиентом и сервером. Данные просто отправляются в виде независимых пакетов (датаграмм) по сети.

Преимущества UDP:

- быстрота: Отсутствие установления соединения и подтверждения доставки делает UDP быстрее, чем TCP;
- меньшая нагрузка: Меньше служебной информации (overhead) в заголовках пакетов.

Недостатки UDP:

- ненадежность: Данные могут быть потеряны, дублированы или доставлены в неправильном порядке. UDP не предоставляет механизмов для восстановления потерянных пакетов или упорядочивания их;
- отсутствие гарантии доставки: Нельзя быть уверенным, что пакет достигнет получателя.

Создание UDP сокетов в Python

Для работы с UDP в Python, как и с TCP, используется модуль socket.

socket.socket(socket.AF_INET, socket.SOCK_DGRAM):

```
import socket
```

```
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
```

Эта строка создает UDP-сокеты.

socket.AF_INET: Указывает на использование IPv4 адресов.

socket.SOCK_DGRAM: Указывает на использование UDP протокола. DGRAM расшифровывается как Datagram, что отражает суть UDP.

Серверная часть (UDP)

```
socket.bind((host, port)):
```

```
HOST = 'localhost'
```

```
PORT = 12345
s.bind((HOST, PORT))
```

Привязывает сокет к определенному адресу и порту. Сервер будет прослушивать входящие UDP-пакеты на этом адресе и порту.

```
socket.recvfrom(buffer_size):
BUFFER_SIZE = 1024
data, address = s.recvfrom(BUFFER_SIZE)
print(f'Получено сообщение '{data.decode('utf-8')}' от {address}')
```

Получает данные и адрес клиента, отправившего данные.

data: Полученные данные (в виде байтов).

address: Адрес клиента (IP-адрес и номер порта).

buffer_size: Максимальный размер буфера для приема данных. Важно выбрать размер, достаточный для приема ожидаемых UDP-пакетов.

```
socket.sendto(data, address):
message = "Эхо: " + data.decode('utf-8')
s.sendto(message.encode('utf-8'), address)
```

Отправляет данные клиенту по указанному адресу. Адрес клиента был получен при вызове recvfrom().

```
socket.close():
```

```
s.close()
```

Закрывает сокет.

UDP предоставляет более быстрый способ передачи данных без гарантий доставки. Это делает его подходящим для приложений реального времени.

UDP-сервер

```
import socket
server_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
#Создание UDP сокета
server_socket.bind(('localhost', 54321)) # Привязка к адресу
print("UDP-сервер запущен...")
while True:
    data, addr = server_socket.recvfrom(1024) # Получение данных и адреса
    отправителя
    print(f'Получено сообщение от {addr}: {data.decode('utf-8')}')
    server_socket.sendto("Ответ от сервера".encode('utf-8'), addr) # Отправка
    ответа
```

Клиентская часть (UDP)

```
socket.sendto(data, (host, port)):
HOST = 'localhost'
PORT = 12345
MESSAGE = "Привет, UDP сервер!"
s.sendto(MESSAGE.encode('utf-8'), (HOST, PORT))
```

Отправляет данные серверу. Указывается адрес и порт сервера. Здесь не нужно предварительно устанавливать соединение.


```

socket.recvfrom(buffer_size) (опционально):
BUFFER_SIZE = 1024
data, address = s.recvfrom(BUFFER_SIZE)
print(f"Получено сообщение '{data.decode('utf-8')}' от {address}")

```

Этот вызов необязателен. Он позволяет клиенту получить ответ от сервера. Например, в нашем эхо-сервере клиент получает повторенное сообщение от сервера. Обратите внимание, что клиент не обязан получать ответ. UDP позволяет отправлять данные в одном направлении.

```

socket.close():
s.close()

```

Закрывает сокет.

UDP-клиент

```

import socket
# Создание UDP клиента
client_socket = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
# Отправка данных
client_socket.sendto("Привет, UDP-сервер!".encode('utf-8'), ('localhost',
54321))
# Получение ответа
data, addr = client_socket.recvfrom(1024)
print(f"Ответ сервера: {data.decode('utf-8')}")
client_socket.close()

```

UDP использует методы `sendto()` и `recvfrom()` вместо `send()` и `recv()`. Это связано с тем, что UDP не устанавливает постоянное соединение между клиентом и сервером.

Примеры кода UDP Socket на Python

Простой UDP-сервер, повторяющий полученные данные:

server.py:

```

import socket
HOST = 'localhost'
PORT = 12345
BUFFER_SIZE = 1024
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.bind((HOST, PORT))
print(f"UDP сервер запущен на {HOST}:{PORT}")
while True:
    data, address = s.recvfrom(BUFFER_SIZE)
    message = "Эхо: " + data.decode('utf-8')
    s.sendto(message.encode('utf-8'), address)
    print(f"Отправлено эхо-сообщение '{message}' клиенту {address}")

```

client.py:

```

import socket
HOST = 'localhost'

```

```

PORT = 12345
MESSAGE = "Привет, UDP сервер!"
BUFFER_SIZE = 1024
s = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
s.sendto(MESSAGE.encode('utf-8'), (HOST, PORT))
data, address = s.recvfrom(BUFFER_SIZE) # Ждем ответа от сервера
print(f"Получено сообщение '{data.decode('utf-8')}' от {address}")
s.close()

```

В коде выше мы реализовали простой UDP-клиент, отправляющий сообщение на сервер. Если вам нужен клиент, который просто отправляет сообщение и не ждет ответа, уберите строки с `recvfrom`.

Пример решения задачи на языке программирования Python

Задача. Напишите консольное клиент-серверное приложение на основе UDP-сокетов, в котором клиент отправляет серверу строку текста. Сервер принимает сообщение, подсчитывает количество слов в полученной строке и отправляет результат обратно клиенту. Клиент должен вывести полученное количество слов на экран.

Решение задачи:

Сервер (server.py)

```

Сервер (udp_server.py)
import socket
server = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
server.bind(('localhost', 5000))
print("UDP-сервер запущен")
while True:
    data, address = server.recvfrom(1024)
    message = data.decode()
    print("Получено:", message)
    count = len(message.split())
    server.sendto(str(count).encode(), address)

```

Клиент (udp_client.py)

```

import socket
client = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
message = input("Введите строку: ")
client.sendto(message.encode(), ('localhost', 5000))
data, address = client.recvfrom(1024)
print("Количество слов:", data.decode())
client.close()

```